

AI ENGINEERING MASTERCLASS · CLASS 1 OF 20

Welcome.
Let us start building.

Ten weeks. Twenty classes. Today you leave with a working, cost-metered LLM call and an environment ready for all of them.

Tahmid Rahman · AI Lead Engineer & Consultant · aiwithtahmid.com

WHY YOU ARE HERE

Two things that are true at the same time

#1

Fastest-growing job title in the world this year: AI Engineer

LinkedIn Jobs on the Rise, January 2026

−20%

Fall in developer jobs for people aged 22 to 25 since 2022

Stanford AI Index, reported April 2026

The work that is shrinking is boilerplate. The work that is growing is judgement: choosing systems, deciding what "working" means, owning cost, reliability and security.
That second half is what this course teaches. Writing Python is table stakes now, not a differentiator.

AGENDA

Today, in ninety minutes

0:00	Why you are here	5 min
0:05	What you will learn, and how this course works	20 min
0:25	The 2026 model landscape, and how to read it	15 min
0:40	**Live build: your first cost-metered LLM call**	25 min
1:05	Assignment and next class	5 min
1:10	Open Q and A	20 min

Every class has this shape: a short concept, then a long build. Nobody watches slides for ninety minutes.

WHAT YOU WILL LEARN

You are not paid to train models

You are paid to decide:

- **Which model, at what cost** — and whether the cheap one is good enough
- **What goes into the context** — and, harder, what stays out of it
- **Whether the output is actually good** — measured, not eyeballed
- **What happens when it is wrong** — because it will be
- **What the system is allowed to do** — and whose authority it acts with

Each of those five is a module of this course. Here is the whole map.

THE PATH

Four modules, four things you can do

01	**Foundations** Classes 1-6	Control a model, engineer its context, and prove it works with numbers
02	**Your data** Classes 7-11	Make it answer from your documents, and measure whether it answered well
03	**Agents** Classes 12-16	Make it take actions in real systems through tools and MCP
04	**Production** Classes 17-20	Make it safe, observable, cheap, and live on a public URL

Classes 1 to 11

1	**AI Engineering in 2026 and the Model Landscape**	Today. The landscape, reading a model card, your first metered call
2	**AI IDE Mastery and Agentic Coding**	Cursor, Antigravity, Claude Code, and reviewing what an agent writes
3	**Model Internals and Model Selection**	Tokens, embeddings, attention, and the Bangla token tax
4	**Generation Control and Production Prompting**	Structured outputs, tool calling, prompts as versioned files
5	**Context Engineering**	Context rot, compaction, caching, and what it costs
6	**Lab 1: Evaluation and Error Analysis**	Ship a tool, build an eval set, read traces by hand
7	**Embeddings, Semantic Search and Vector Databases**	A vector store in fifty lines, then a real database
8	**RAG Fundamentals and Measurement**	Retrieve, augment, generate, and score the retrieval
9	**Advanced Retrieval and GraphRAG**	Hybrid search, rerankers, knowledge graphs, agentic search
10	**Document AI, Bangla and LLM-as-Judge**	Messy PDFs, OCR, and a judge you have calibrated
11	**Lab 2: Domain Knowledge Assistant**	Ingestion through evaluation, inside a cost budget

Dark numbers are assessed labs. You hand in working code and the report that proves it works.

Classes 12 to 20

12	**AI Agents and Tool Design**	The loop in forty lines, then tools the model can recover from
13	**Agent Frameworks, Memory and Long-Horizon Work**	SDKs, sessions, checkpoints, and agents that remember you
14	**Model Context Protocol (MCP)**	Use servers you did not write, then publish your own
15	**Multi-Agent Systems and Computer Use**	Orchestration, browser agents, and when multi-agent fails
16	**Lab 3: Production Agent**	A database, voice or workflow agent, scored on its trajectory
17	**AI Security, Guardrails and Reliability**	Prompt injection, the lethal trifecta, sandboxing, guardrails
18	**Observability, Serving and Cost Optimisation**	Tracing, quantisation, caching, routing, and halving your bill
19	**Deployment and Fine-Tuning Decisions**	FastAPI, Docker, spend caps, and when fine-tuning actually pays
20	**Capstone Demo Day and Career**	Present, get assessed, and position yourself for the job

By Class 20 every one of these has produced something in your repository.

WHAT YOU WILL HAVE BUILT

Six things you can show an employer

Class 6

An evaluated LLM tool

shipped with the eval suite that proves it behaves

Class 8

A citing RAG system

retrieval over real documents, with a measured scorecard

Class 11

A Bangla document assistant

ingestion through evaluation, inside a cost budget

Class 14

A published MCP server

infrastructure other people’s agents can use

Class 16

An agent that does a real job

scored on how it got there, not just the answer

Class 20

Your capstone

deployed publicly, secured, traced and cost-optimised

How you are assessed

Every week	A build assignment, reviewed by your pair, with my spot-checks
Classes 6, 11, 16	Three assessed labs, marked on working code and its evaluation report
Class 20	Capstone, presented on Demo Day and marked against a published rubric
Certificate	Awarded on the capstone. Attendance alone does not earn one

You are paired in week one. Your pair reviews your code, you review theirs.

HOW THIS COURSE WORKS

This course costs you nothing to run

Every class is designed against free provider tiers, open-weight models you host yourself, and free cloud notebooks. There is no paid API or cloud spend at any point in the twenty classes.

What you need

A laptop
Basic Python
Free accounts, set up today

What you do not need

A GPU
A paid API key
Any machine learning background

What that costs me

Designing every lab to fit inside free limits. That constraint is part of the course.

If a class would ever need paid access, it gets redesigned around a smaller local model instead.

HOW TO GET THROUGH THIS COURSE

Your week, every week

1

In class

You build along, on your own machine. Watching is not learning. If your code does not run, say so in the room.

2

Between classes

Two to three hours on the extension assignment. This is where the learning actually happens.

3

With your pair

You review their code, they review yours. You will learn as much from reading as from writing.

The one rule: never stay stuck for more than a day.

Ask the group first, then your pair, then me. Being stuck quietly for a week is the only way to fall behind in this course.

THE 2026 LANDSCAPE

Three families of model, three reasons to pick one

Frontier

Closed weights, used through an API

Best capability per call. You pay per token and cannot see inside.

Pick when the task is hard and volume is low

Open-weight

Downloadable, run anywhere

Near-frontier on many tasks now. Free to run on your own machine.

Pick for volume, privacy, or zero marginal cost

Reasoning

Spends compute before answering

Thinks in tokens you pay for. The budget is a dial you control.

Pick for multi-step problems, not for extraction

Model names change every few months. The three families, and the reasons to choose between them, do not.

THE 2026 LANDSCAPE

Reasoning models and thinking budgets

The model generates hidden tokens working the problem out before it answers you. You are billed for those tokens.

It earns its cost

Multi-step problems
Code that must compile
Anything needing verification
Ambiguous instructions

It burns money

Structured extraction
Classification
Formatting and rewriting
Anything you could regex

MEASURED IN TODAY'S BUILD · same model, same prompt, same 300-token ceiling

```
effort=low     126 tokens     answered          $0.000044
effort=high    300 tokens     EMPTY ANSWER      $0.000096
```

Twice the price to return nothing at all. The budget is a dial, not a switch.

THE SKILL, NOT THE SNAPSHOT

How to read a model card

Look for

Context window, in and out
Modalities it accepts
Knowledge cutoff
Tool and structured-output support
Licence, for open weights
Which evals, on what data

Ignore

"Most capable model ever"
A benchmark with no baseline
A chart with no axis labels
Comparisons against last year
Anything without a number

We are about to open a real one together. Bring it up on your own screen as I do.

How to read a pricing page: three numbers

Input

What you send

Cheapest of the three

Output

What it sends back

Usually 3 to 5 times input

Cached input

What you resend

Often about a tenth of input

Worked example · example rates of $3 per M in, $15 per M out

2,000-token prompt + 500-token answer, run 1,000 times

in: 2,000 × 1,000 = 2M tokens → $6.00 out: 500 × 1,000 = 0.5M tokens → $7.50 total $13.50

Output is a third of the tokens and more than half the bill. Answer length is a cost decision.

Thinking tokens bill as output. These rates are an example, not a quote. In the build we pull the live price list straight from the API, which is the only version that is never stale.

LIVE BUILD · 25 MINUTES

Your first LLM call, with a cost counter and a hard spending cap

By the end of this block your script answers a question, prints the tokens it used and what they cost, and refuses to run at all if the call would push you past your budget.

Everyone gets this working before we move on. Raise your hand early, not at the end.

LIVE BUILD

Step 1 · Accounts, environment, repo

1

Create your free Groq account and key

Groq, with a q. It serves open-weight models fast, on a free tier. Not Grok, the xAI model.

2

Create the project and virtual environment

One folder for the whole course. You use it in all twenty classes.

3

Keys in a .env file, never in code

And .env in .gitignore before your first commit.

4

Initialise the repository and push

This repo becomes your portfolio by Class 20.

```
Groq keys start with gsk_ and talk to api.groq.com. Grok keys start with xai-. Read the prefix, not the brand you remember.
```

LIVE BUILD

Step 2 · Call, count, cost

Three things every call in this course will report back.

```
r = client.chat.completions.create(model=MODEL, max_tokens=512, messages=[...])

u    = r.usage
cost = u.prompt_tokens/1e6*RATE_IN + u.completion_tokens/1e6*RATE_OUT
spend.total += cost

print(f"{u.prompt_tokens} in  {u.completion_tokens} out  ${cost:.8f}")
```

What it actually printed

`103 in  130 out  $0.00004672`

Tokens are the unit. Not words, not characters.

Now multiply it

`1,000 → $0.05` `1,000,000 → $46.72`

A tiny number times a user base is a real bill.

LIVE BUILD

Step 3 · The guard that refuses to spend

A cost counter tells you what you spent. A guard stops you spending it. Estimate before the call, not after.

```
BUDGET = 0.05          # dollars, for the whole session

def guarded_call(messages, max_out, model=MODEL):
    est = estimate_cost(messages, max_out, *rates(model))
    if spend.total + est > BUDGET:
        raise RuntimeError(f"Refused. {spend.total:.6f} + {est:.6f} > {BUDGET}")
    r = client.chat.completions.create(...)
    return r, r.choices[0].finish_reason        # always read this
```

Set the cap twice

In code, as above, and as a hard limit in the provider dashboard. Code you can edit by accident at 2am; the dashboard limit you cannot.

finish_reason == 'length'

Means you cut the answer off and paid in full for it. On a reasoning model you can get an empty string back. It is never an error, so you must check.

BEFORE THE NEXT CLASS

Your first assignment

Run the same prompt against three models. Push the notebook with your outputs visible, class1_results.json, and a three-line README: which model you would put in production, what it costs per thousand requests, and why you rejected the other two.

Next class

Class 2: AI IDE Mastery. Cursor, Antigravity and agentic coding. Come with your repo working.

The one thing to remember

The README is the marked part. Anyone can run three calls. The judgement is the deliverable.